

Python 数据分析课程期末作业

豆瓣电影 Top200

数据爬取、清洗、分析与可视化

基于 Python 的公开电影榜单数据探索性分析报告

课程名称：Python 网络爬虫与信息爬取

学生姓名：刘智鹏

学号：202424073122

班级：24 信息 1 班

完成日期：2026 年 6 月

目录

摘要与关键词

1 研究背景与目标

2 数据来源与爬取方法

3 数据清洗与质量核验

4 探索性分析与可视化

5 结论

6 局限性

参考文献

附录 A 核心 Python 代码

摘要

本研究以豆瓣电影 Top250 公开列表页中的前 200 名为数据来源，使用 Python 完成分页数据采集、数据清洗、探索性分析和静态可视化。本次运行共获得 200 条原始记录，清洗后保留 200 条有效电影记录。样本平均评分为 9.01 分，中位数为 9.00 分，覆盖 1936 至 2023 年上映的作品。研究从类型、国家/地区、年代和用户评价规模等角度解释榜单构成，为理解高口碑电影的共同特征提供数据支持。

关键词：Python；网络爬虫；数据清洗；探索性分析；数据可视化；电影数据

1 研究背景与目标

在线电影榜单同时包含作品属性、公众评分与评价人数等信息，是练习网络数据采集和探索性分析的合适场景。本作业以公开榜单页面为对象，避免登录、验证码绕过和高频访问，仅获取列表页中已公开展示的信息。

研究目标包括：第一，构建可复跑的 Python 数据管道；第二，通过规范化清洗使多字段文本能够用于统计；第三，比较榜单中不同类型、国家/地区和年代作品的分布；第四，观察评分、评价人数与榜单排名之间的关系。

2 数据来源与爬取方法

数据来自豆瓣电影 Top250 公开列表页 (<https://movie.douban.com/top250>)，本研究取其中排名第 1 至 200 名的电影作为 Top200 样本。本次数据处理日期为 2026-06-26。程序依次访问 8 个分页，每页解析 25 条记录，采集排名、片名、原始片名、导演演员原始信息、上映年份、国家/地区、类型、评分、评价人数、短评和详情页链接。

该样本是榜单前列作品构成的非随机高口碑样本，适合描述榜单内部特征，不用于代表全部电影市场。为控制访问压力，程序在页与页之间随机等待 1.2 至 2.0 秒，对单页请求最多重试 3 次，并把每次请求结果写入 JSONL 日志。原始 HTML 页面和原始 CSV 会被保留；使用 `--skip-fetch` 时，程序不访问网络，而是基于已保存页面重新解析前 200 名数据，并生成明确标注为“离线复核”的 Top200 日志。全流程仅用于课程学习，未登录、未绕过验证码或访问限制。

3 数据清洗与质量核验

清洗阶段首先将空白与缺失值统一为空字符串，避免将缺失原始片名错误保存为字符串“nan”；随后把排名、年份、评分和评价人数转换为数值类型，并验证评分范围、评价人数非负、排名唯一和年份合理性。导演和演员信息字段（people_info）保留列表页摘要原文，部分记录会以省略号截断；该字段仅用于原始信息追溯，不参与本报告的统计分析或可视化。对于国家/地区与类型等多值字段，主表保留原始组合文本，分析时另外拆分为一对多表，避免改变电影主表的记录粒度。

3.1 原始数据样例

表 1 展示本次输入数据的前 5 条记录及主要字段；其中“—”表示网页中未提供原始片名，而不是字符串 nan。

排名	中文片名	原始片名	年份	国家/地区	评分
1	肖申克的救赎	The Shawshank Redemption	1994	美国	9.7
2	霸王别姬	—	1993	中国大陆 / 中国香港	9.6
3	泰坦尼克号	Titanic	1997	美国	9.5
4	阿甘正传	Forrest Gump	1994	美国	9.5
5	千与千寻	千と千尋の神隠し	2001	日本	9.4

表 1 原始数据样例（前 5 条）

3.2 数据质量核验

核验指标	结果
原始记录数	200
无效记录剔除数	0
重复排名剔除数	0
重复片名/年份剔除数	0

清洗后主表记录数	200
清洗后缺失上映年份数	0
评分范围	8.5 - 9.7
排名是否唯一	是

表 2 数据清洗与质量核验结果（由本次程序运行自动生成）

4 探索性分析与可视化

4.1 评分分布

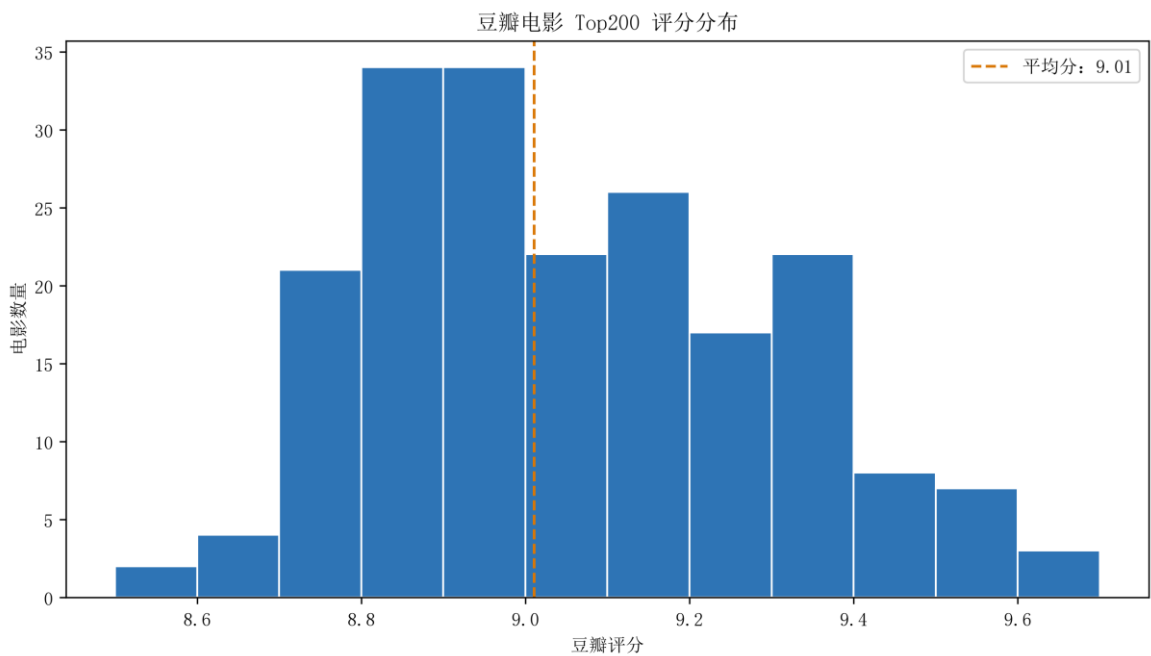


图 1 Top200 电影评分分布

数据来源：本研究基于豆瓣电影 Top250 公开列表页中排名前 200 名电影整理。

样本评分集中在较高区间，平均值为 9.01 分，说明该榜单筛选出的作品整体具有较高口碑。

4.2 热门电影类型

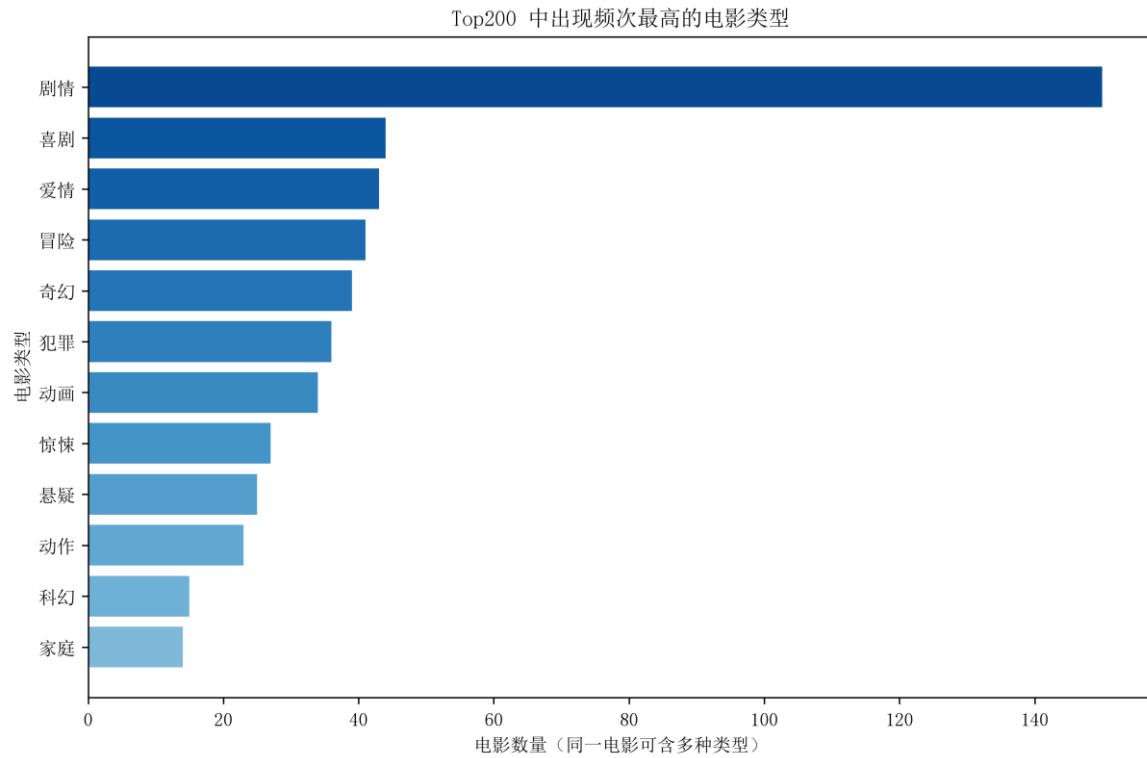


图 2 Top200 中出现频次最高的电影类型

数据来源：本研究基于豆瓣电影 Top250 公开列表页中排名前 200 名电影整理。

出现最多的类型是“剧情”，共出现 150 次。由于一部电影可以具有多种类型，图中的总次数大于电影总数。

4.3 主要制片国家/地区

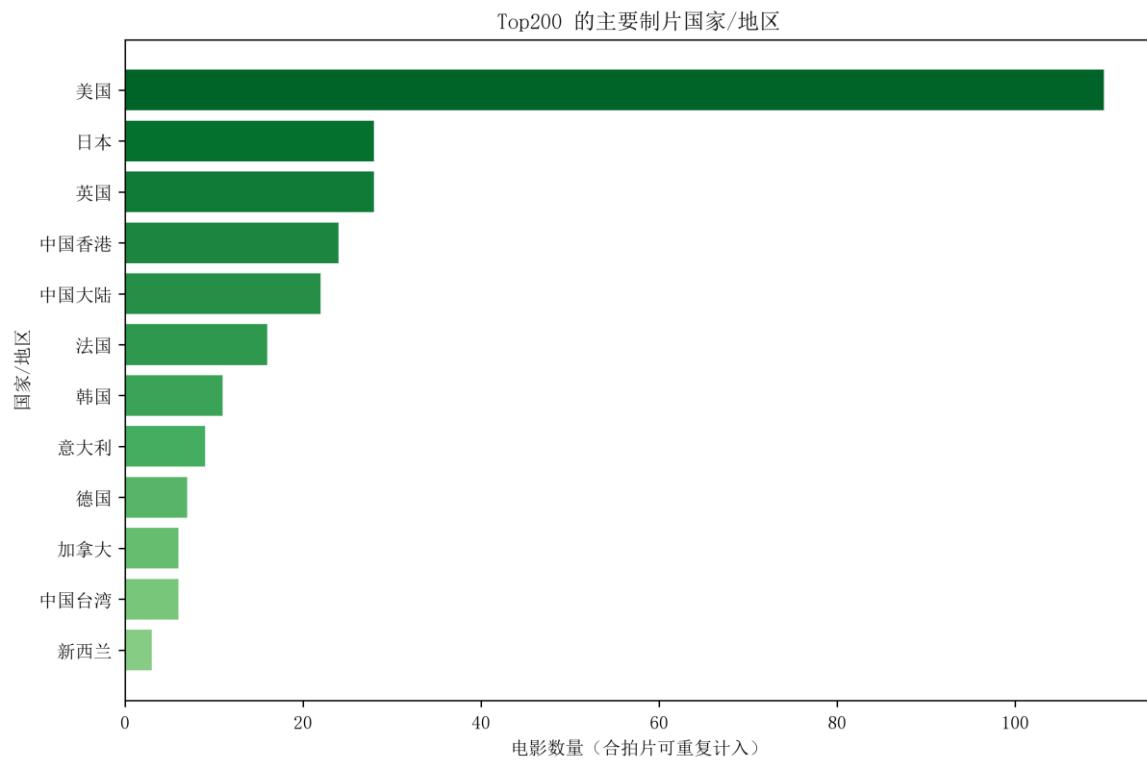


图 3 Top200 的主要制片国家/地区

数据来源：本研究基于豆瓣电影 Top250 公开列表页中排名前 200 名电影整理。

“美国”出现 110 次，是样本中出现频率最高的制片国家/地区。合拍片会分别计入其对应国家/地区。

4.4 年代分布

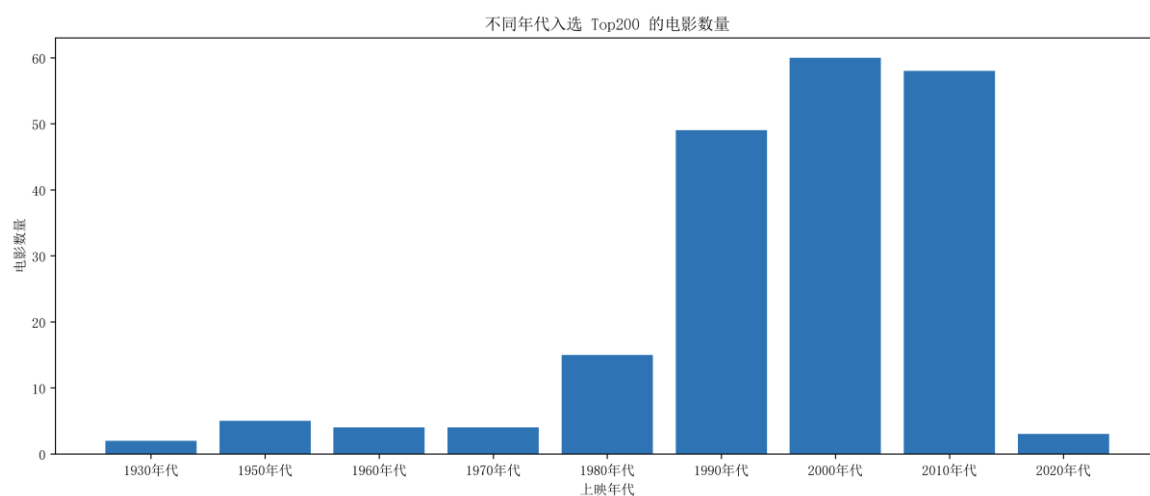


图 4 不同年代入选 Top200 的电影数量

数据来源：本研究基于豆瓣电影 Top250 公开列表页中排名前 200 名电影整理。

入选作品最多的是 2000 年代，共 60 部，反映了该年代作品在榜单中的集中程度。

4.5 年代与平均评分

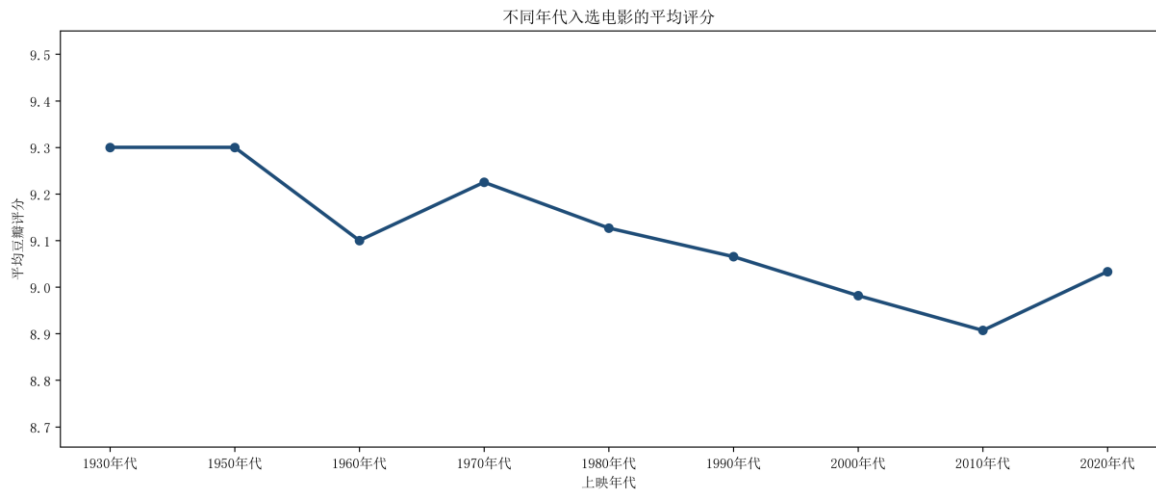


图 5 不同年代入选电影的平均评分

数据来源：本研究基于豆瓣电影 Top250 公开列表页中排名前 200 名电影整理。

在样本覆盖的年代中，1930 年代的平均评分最高，约为 9.30 分；但该年代仅有 2 部入选样本，样本量较小，均值仅供参考。该结果仅描述入选榜单的电影，不代表该年代所有电影的总体质量。

4.6 评分与评价人数

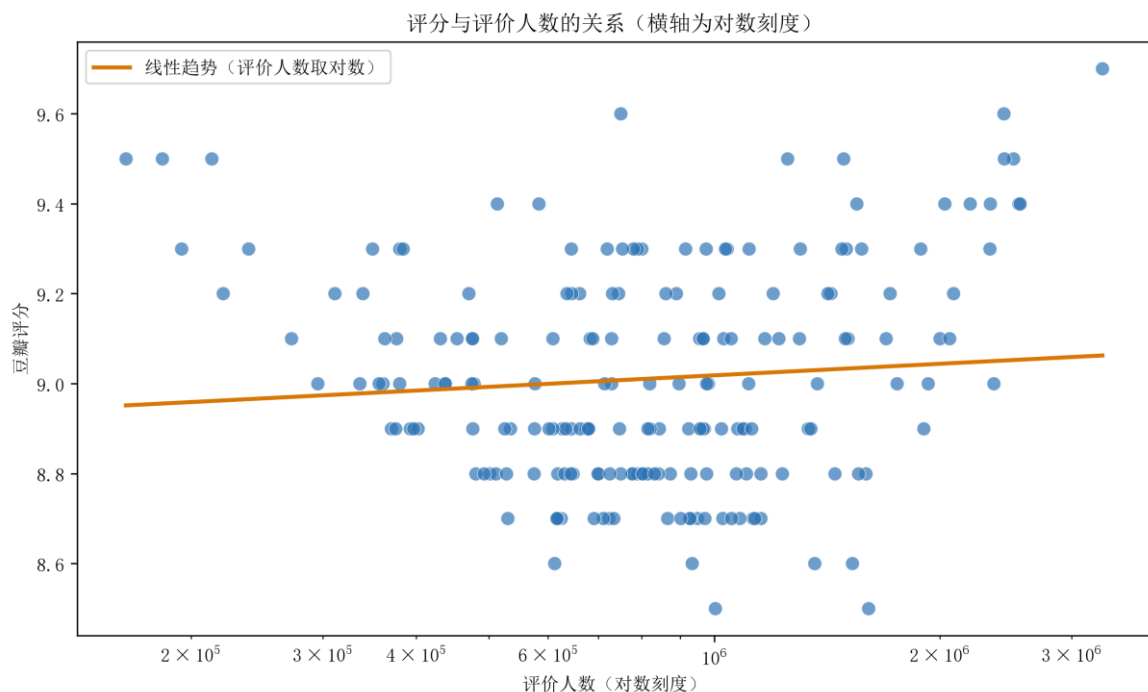


图 6 评分与评价人数的关系

数据来源：本研究基于豆瓣电影 Top250 公开列表页中排名前 200 名电影整理。

评分与评价人数对数的 Pearson 相关系数为 0.085 ($p=0.234$)，Spearman 秩相关系数为 0.073 ($p=0.307$)。两种结果均表明二者在本样本中仅呈极弱关系；图中的趋势线仅用于辅助观察，不代表因果关系。横轴取对数，是为了同时展示评价人数差异较大的作品。

4.7 排名与评分

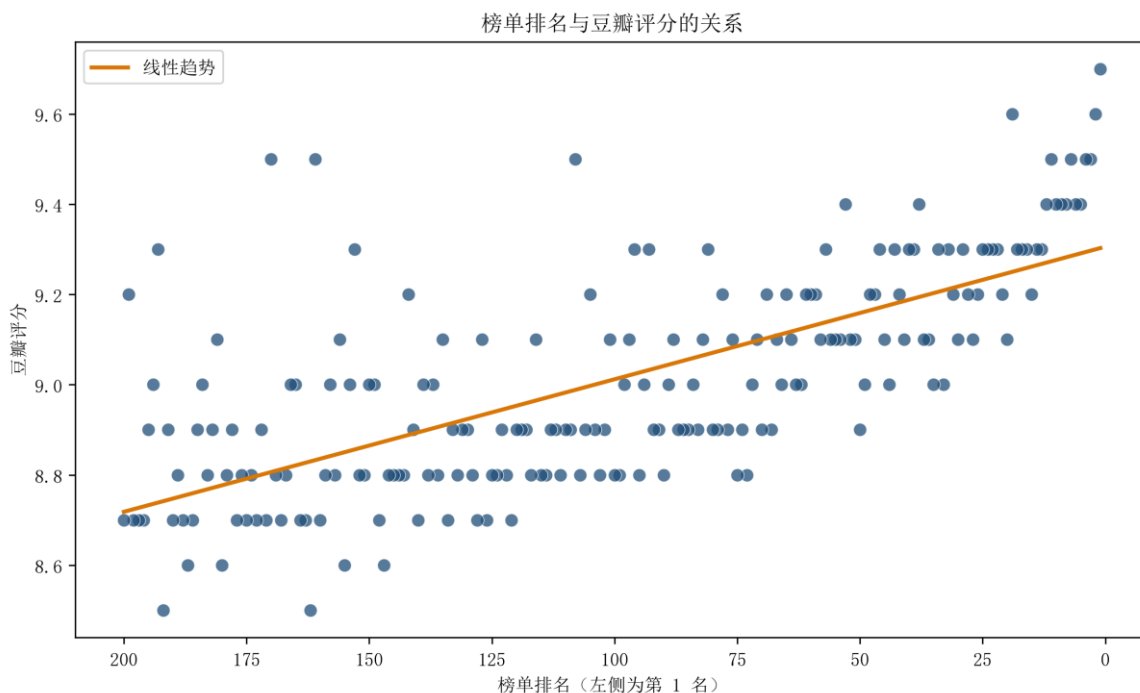


图 7 榜单排名与豆瓣评分的关系

数据来源：本研究基于豆瓣电影 Top250 公开列表页中排名前 200 名电影整理。

排名与评分的 Spearman 秩相关系数为 -0.706 ($p < 0.001$)。由于排名数字越小表示位置越靠前，负相关意味着较高评分通常对应更靠前的排名；趋势线用于展示总体方向，具体排名还会受到评分精度和评价人数等因素影响。

5 结论

(1) 本项目成功构建了从网页采集到报告输出的 Python 自动化流程。清洗后 200 条记录的排名均唯一，评分范围为 8.5 至 9.7 分，数据质量满足本次探索性分析需要。

(2) Top200 呈现出明显的多类型特征，其中“剧情”最常见；国家/地区方面，“美国”出现频次最高。两项分布均需结合榜单的口碑筛选性质理解，不能外推到全球全部电影市场。

(3) 时间维度上，2000 年代入选作品最多，而 1930 年代的样本平均评分最高。评分与评价人数对数的 Pearson 相关系数仅为 0.085 ($p=0.234$)，说明在该 Top200 样本中没有明显的线性关联；排名与评分的秩相关更强，但这仅描述榜单内部的排序关系，不能据此推断因果。上述显著性检验用于辅助判断本样本内关系的稳定性，不应被解释为对全部电影总体的推断。

6 局限性

第一，数据来自一个动态更新的公开榜单，抓取日期不同可能导致评分和评价人数变化。第二，样本为豆瓣 Top250 榜单的前 200 名，是非随机的高口碑选择样本，不能代表全部电影或电影市场。第三，国家/地区与类型来自页面文本拆分，合拍片和多类型电影会在展开统计中重复计入。第四，本项目的相关与显著性检验只用于描述和辅助解释本样本内的关系，不将相关关系解释为因果关系，也不将其外推为总体结论。

参考文献

- [1] 豆瓣电影. 豆瓣电影 Top250 (本研究取排名前 200 名) [EB/OL]. <https://movie.douban.com/top250> (访问日期: 2026-06-26).
- [2] 张良均, 刘名奇, 龚志军, 等. Python 数据分析与挖掘实战 (第 2 版) [M]. 北京: 机械工业出版社, 2019.
- [3] 李航. 统计学习方法 (第 2 版) [M]. 北京: 清华大学出版社, 2019.
- [4] McKinney W. Python for Data Analysis[M]. 3rd ed. O'Reilly Media, 2022.
- [5] Hunter J D. Matplotlib: A 2D Graphics Environment[J]. Computing in Science & Engineering, 2007, 9(3): 90-95.

附录 A 核心 Python 代码

以下代码节选来自本项目源码，用于展示页面解析、低频分页采集、数据清洗、统计分析和可视化逻辑。完整可运行代码见项目的 src 目录；附录代码字号为 9.5 pt。

A.1 单页 HTML 解析函数

```
def parse_top250_html(html: str, source_url: str, fetched_at: str) -> list[dict[str, Any]]:
    """解析一个榜单页面，返回结构统一的一电影一行记录。"""
    soup = BeautifulSoup(html, "html.parser")
    records: list[dict[str, Any]] = []

    for item in soup.select("ol.grid_view > li > div.item"):
        rank_tag = item.select_one(".pic em")
        title_tags = item.select(".hd .title")
        # 豆瓣当前页面的评分容器不保证带有 .star 类，评分数字本身更稳定。
        rating_tag = item.select_one(".rating_num")
        info_tag = item.select_one(".bd > p")
        link_tag = item.select_one(".hd a[href]")
        if not all([rank_tag, title_tags, rating_tag, info_tag, link_tag]):
            raise CrawlError("页面中存在无法识别的电影条目，已停止以避免生成不完整数据。")

        titles = [_clean_text(tag.get_text(" ", strip=True)) for tag in title_tags]
        title = titles[0]
        original_title = " / ".join(titles[1:]).rstrip("/")
        people_info, year, country_text, genre_text = _parse_info_text(info_tag.get_text(" ",
strip=True))

        star_text = _clean_text(rating_tag.parent.get_text(" ", strip=True))
        voters_match = re.search(r"([\d,]+)\s*人评价", star_text)
        quote_tag = item.select_one(".quote span")
        records.append(
            {
                "rank": int(rank_tag.get_text(strip=True)),
                "title": title,
                "original_title": original_title,
                "people_info": people_info,
                "year": year,
                "country_text": country_text,
                "genre_text": genre_text,
                "rating": float(rating_tag.get_text(strip=True)),
                "rating_count": int(voters_match.group(1).replace(",", "")) if voters_match
            }
        )
        if quote_tag:
            records[-1]["quote"] = _clean_text(quote_tag.get_text(" ", strip=True)) if quote_tag else ""
            records[-1]["detail_url"] = link_tag["href"]
            records[-1]["source_url"] = source_url
            records[-1]["fetched_at"] = fetched_at,
    )
    return records
```

A.2 分页采集函数

```
def crawl_top200(raw_csv_path: Path, pages_dir: Path, log_path: Path, settings: CrawlSettings |
None = None) -> list[dict[str, Any]]:
    """以低频率爬取豆瓣 Top250 原榜的前 200 名，保存网页、CSV 与 JSONL 日志。"""
    settings = settings or CrawlSettings()
    raw_csv_path.parent.mkdir(parents=True, exist_ok=True)
    pages_dir.mkdir(parents=True, exist_ok=True)
    log_path.parent.mkdir(parents=True, exist_ok=True)
    log_path.write_text("", encoding="utf-8")

    session = requests.Session()
    session.headers.update(
        {
            "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
            "(KHTML, like Gecko) Chrome/124 Safari/537.36 CourseDataAnalysis/1.0",
            "Accept-Language": "zh-CN,zh;q=0.9,en;q=0.7",
        }
    )
    fetched_at = utc_now()
    records: list[dict[str, Any]] = []

    for page_number, start in enumerate(range(0, TOP_N, PAGE_SIZE), start=1):
        url = f"{BASE_URL}?start={start}&filter="
        html = _request_page(session, url, settings, log_path, start)
        (pages_dir / f"top200_{page_number:02d}.html").write_text(html, encoding="utf-8")
        page_records = parse_top250_html(html, url, fetched_at)
```

```

    if len(page_records) != PAGE_SIZE:
        raise CrawlError(f"榜单第 {page_number} 页解析到 {len(page_records)} 条, 预期为 {PAGE_SIZE} 条。")
    records.extend(page_records)
    _write_log(
        log_path,
        {"timestamp": _utc_now(), "event": "parse_ok", "page": page_number, "start": start,
         "records": len(page_records)},
    )
    if start < TOP_N - PAGE_SIZE:
        time.sleep(random.uniform(settings.min_delay_seconds, settings.max_delay_seconds))

    ranks = [record["rank"] for record in records]
    if len(records) != TOP_N or len(set(ranks)) != TOP_N or set(ranks) != set(range(1, TOP_N + 1)):
        raise CrawlError(f"爬取完成后的记录校验失败: 记录数={len(records)}, 唯一排名数={len(set(ranks))}。")

    _write_raw_csv(records, raw_csv_path)
    _write_log(log_path, {"timestamp": _utc_now(), "event": "crawl_complete", "records": len(records),
                          "raw_csv": str(raw_csv_path)})
    return records

```

A.3 清洗函数

```

def clean_movies(raw_df: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, dict[str, Any]]:
    """清洗原始主表, 并构造国家/地区、类型的展开分析表。"""
    required_columns = {"rank", "title", "year", "rating", "rating_count", "country_text", "genre_text"}
    missing_columns = required_columns.difference(raw_df.columns)
    if missing_columns:
        raise ValueError(f"原始数据缺少字段: {' , '.join(sorted(missing_columns))}")

    working = raw_df.copy()
    for column in TEXT_COLUMNS:
        if column in working.columns:
            working[column] = working[column].map(clean_text)
    for column in ["rank", "year", "rating", "rating_count"]:
        working[column] = pd.to_numeric(working[column], errors="coerce")

    input_records = len(working)
    missing_required = working[["rank", "title", "rating"]].isna().any(axis=1) | (working["title"] == "")
    current_year = datetime.now().year
    invalid_range = (
        working["rating"].lt(0)
        | working["rating"].gt(10)
        | working["rating_count"].lt(0)
        | working["rank"].lt(1)
        | working["year"].notna() & (working["year"].lt(1888) | working["year"].gt(current_year))
    )
    invalid_rows = missing_required | invalid_range
    dropped_invalid = int(invalid_rows.sum())
    working = working.loc[~invalid_rows].copy()

    duplicate_rank = int(working.duplicated(subset=["rank"], keep="first").sum())
    working = working.drop_duplicates(subset=["rank"], keep="first")
    duplicate_title_year = int(working.duplicated(subset=["title", "year"], keep="first").sum())
    working = working.drop_duplicates(subset=["title", "year"], keep="first")
    working = working.sort_values("rank").reset_index(drop=True)
    working["rank"] = working["rank"].astype(int)
    working["rating"] = working["rating"].astype(float)
    working["rating_count"] = working["rating_count"].fillna(0).astype(int)
    working["year"] = working["year"].astype("Int64")
    working["decade"] = (working["year"] // 10 * 10).astype("Int64")

    country_df = (
        working.assign(country=working["country_text"].map(lambda value:
            [normalize_country_name(item) for item in split_multi_value(value)]))
        .explode("country")
        .dropna(subset=["country"])
        .query("country != ''")
        [["rank", "title", "year", "rating", "rating_count", "country"]]
    )

```

```

        .reset_index(drop=True)
    )
    genre_df = (
        working.assign(genre=working["genre_text"].map(split_multi_value))
        .explode("genre")
        .dropna(subset=["genre"])
        .query("genre != ','")
        [["rank", "title", "year", "rating", "rating_count", "genre"]]
        .reset_index(drop=True)
    )
    quality = {
        "input_records": input_records,
        "dropped_invalid_records": dropped_invalid,
        "duplicate_rank_removed": duplicate_rank,
        "duplicate_title_year_removed": duplicate_title_year,
        "output_records": int(len(working)),
        "missing_year_after_cleaning": int(working["year"].isna().sum()),
        "missing_country_after_cleaning": int((working["country_text"] == "").sum()),
        "missing_genre_after_cleaning": int((working["genre_text"] == "").sum()),
        "rating_range": [float(working["rating"].min()), float(working["rating"].max())],
        "rank_is_unique": bool(working["rank"].is_unique),
    }
    return working, country_df, genre_df, quality

```

A.4 统计分析函数

```

def build_summary(clean_df: pd.DataFrame, country_df: pd.DataFrame, genre_df: pd.DataFrame,
quality: dict[str, Any]) -> tuple[dict[str, Any], pd.DataFrame]:
    """计算报告和图表共享的统计结果。"""
    genre_counts = genre_df["genre"].value_counts()
    country_counts = country_df["country"].value_counts()
    decade_counts = clean_df.dropna(subset=["decade"]).groupby("decade",
observed=True).size().sort_index()
    decade_ratings = clean_df.dropna(subset=["decade"]).groupby("decade",
observed=True)["rating"].mean().sort_index()
    decade_table = pd.DataFrame(
        {
            "decade": decade_counts.index.astype(int),
            "movie_count": decade_counts.values.astype(int),
            "average_rating": decade_ratings.reindex(decade_counts.index).round(3).values,
        }
    )
    top_movies = clean_df.sort_values(["rating", "rating_count"], ascending=[False,
False]).head(10)
    log_count = (clean_df["rating_count"] + 1).map(math.log10)
    rating_values = clean_df["rating"]
    pearson = stats.pearsonr(rating_values, log_count)
    rating_votes_spearman = stats.spearmanr(rating_values, log_count)
    rank_rating_spearman = stats.spearmanr(clean_df["rank"], rating_values)
    summary = {
        "generated_at": datetime.now().astimezone().isoformat(timespec="seconds"),
        "quality": quality,
        "overall": {
            "movie_count": int(len(clean_df)),
            "average_rating": round(float(clean_df["rating"].mean()), 3),
            "median_rating": round(float(clean_df["rating"].median()), 3),
            "min_rating": round(float(clean_df["rating"].min()), 3),
            "max_rating": round(float(clean_df["rating"].max()), 3),
            "total_rating_count": int(clean_df["rating_count"].sum()),
            "year_min": int(clean_df["year"].dropna().min()),
            "year_max": int(clean_df["year"].dropna().max()),
            "rating_log_count_correlation": round(float(pearson.statistic), 3),
            "rating_log_count_pearson_p_value": float(pearson.pvalue),
            "rating_log_count_spearman_rho": round(float(rating_votes_spearman.statistic), 3),
            "rating_log_count_spearman_p_value": float(rating_votes_spearman.pvalue),
            "rank_rating_spearman_rho": round(float(rank_rating_spearman.statistic), 3),
            "rank_rating_spearman_p_value": float(rank_rating_spearman.pvalue),
        },
        "top_genres": records_from_series(genre_counts, "genre", 12),
        "top_countries": records_from_series(country_counts, "country", 12),
        "decades": decade_table.to_dict(orient="records"),
        "top_movies": [
            {"rank": int(row.rank), "title": row.title, "rating": float(row.rating),
"rating_count": int(row.rating_count)}

```

```

        ],
        for row in top_movies.itertuples(index=False)
    }
    return summary, decade_table

```

A.5 可视化函数

```

def create_figures(clean_df: pd.DataFrame, country_df: pd.DataFrame, genre_df: pd.DataFrame,
figures_dir: Path) -> dict[str, Path]:
    """生成 7 张中文静态图表，返回其语义化路径映射。"""
    font_name = select_chinese_font()
    plt.rcParams.update({"font.family": font_name, "font.sans-serif": [font_name],
"axes.unicode_minus": False, "figure.dpi": 120})
    figures_dir.mkdir(parents=True, exist_ok=True)
    paths = {
        "rating_distribution": figures_dir / "01_评分分布.png",
        "genre_counts": figures_dir / "02_热门电影类型.png",
        "country_counts": figures_dir / "03_主要制片国家地区.png",
        "decade_counts": figures_dir / "04_不同年代电影数量.png",
        "decade_ratings": figures_dir / "05_不同年代平均评分.png",
        "rating_votes": figures_dir / "06_评分与评价人数关系.png",
        "rank_rating": figures_dir / "07_排名与评分关系.png",
    }
    accent, dark, warm = "#2E74B5", "#1F4D78", "#D97706"

    plt.figure(figsize=(9, 5.2))
    plt.hist(clean_df["rating"], bins=12, color=accent, edgecolor="white")
    plt.axvline(clean_df["rating"].mean(), color=warm, linestyle="--", label=f"平均分: {clean_df['rating'].mean():.2f}")
    plt.title("豆瓣电影 Top200 评分分布")
    plt.xlabel("豆瓣评分")
    plt.ylabel("电影数量")
    plt.legend()
    _finish_plot(paths["rating_distribution"])

    top_genres = genre_df["genre"].value_counts().head(12).sort_values()
    plt.figure(figsize=(9, 6))
    plt.barh(top_genres.index, top_genres.values, color=plt.cm.Blues(np.linspace(0.45, 0.9, len(top_genres))))
    plt.title("Top200 中出现频次最高的电影类型")
    plt.xlabel("电影数量 (同一电影可含多种类型)")
    plt.ylabel("电影类型")
    _finish_plot(paths["genre_counts"])

    top_countries = country_df["country"].value_counts().head(12).sort_values()
    plt.figure(figsize=(9, 6))
    plt.barh(top_countries.index, top_countries.values, color=plt.cm.Greens(np.linspace(0.45, 0.9, len(top_countries))))
    plt.title("Top200 的主要制片国家/地区")
    plt.xlabel("电影数量 (合拍片可重复计入)")
    plt.ylabel("国家/地区")
    _finish_plot(paths["country_counts"])

    decade_counts = clean_df.dropna(subset=["decade"]).groupby("decade",
observed=True).size().sort_index()
    plt.figure(figsize=(12, 5.2))
    plt.bar([f"{int(value)} 年代" for value in decade_counts.index], decade_counts.values,
color=accent)
    plt.title("不同年代入选 Top200 的电影数量")
    plt.xlabel("上映年代")
    plt.ylabel("电影数量")
    plt.xticks(rotation=0, ha="center")
    _finish_plot(paths["decade_counts"])

    decade_ratings = clean_df.dropna(subset=["decade"]).groupby("decade",
observed=True)["rating"].mean().sort_index()
    plt.figure(figsize=(12, 5.2))
    plt.plot([f"{int(value)} 年代" for value in decade_ratings.index], decade_ratings.values,
marker="o", color=dark, linewidth=2.5)
    plt.title("不同年代入选电影的平均评分")
    plt.xlabel("上映年代")
    plt.ylabel("平均豆瓣评分")
    plt.ylim(max(0, decade_ratings.min() - 0.25), min(10, decade_ratings.max() + 0.25))
    plt.xticks(rotation=0, ha="center")

```

```

_finish_plot(paths["decade_ratings"])

plt.figure(figsize=(9, 5.5))
log_rating_count = np.log10(clean_df["rating_count"].to_numpy() + 1)
plt.scatter(clean_df["rating_count"], clean_df["rating"], s=58, alpha=0.7, color=accent,
edgecolors="white", linewidths=0.4)
slope, intercept = np.polyfit(log_rating_count, clean_df["rating"], 1)
x_line = np.linspace(log_rating_count.min(), log_rating_count.max(), 100)
plt.plot(10**x_line - 1, slope * x_line + intercept, color=warm, linewidth=2.2, label="线性
趋势（评价人数取对数）")
plt.xscale("log")
plt.title("评分与评价人数的关系（横轴为对数刻度）")
plt.xlabel("评价人数（对数刻度）")
plt.ylabel("豆瓣评分")
plt.legend()
_finish_plot(paths["rating_votes"])

plt.figure(figsize=(9, 5.5))
plt.scatter(clean_df["rank"], clean_df["rating"], color=dark, alpha=0.75, s=52,
edgecolors="white", linewidths=0.4)
rank_slope, rank_intercept = np.polyfit(clean_df["rank"], clean_df["rating"], 1)
rank_line = np.linspace(clean_df["rank"].min(), clean_df["rank"].max(), 100)
plt.plot(rank_line, rank_slope * rank_line + rank_intercept, color=warm, linewidth=2.2,
label="线性趋势")
plt.gca().invert_xaxis()
plt.title("榜单排名与豆瓣评分的关系")
plt.xlabel("榜单排名（左侧为第 1 名）")
plt.ylabel("豆瓣评分")
plt.legend()
_finish_plot(paths["rank_rating"])
return paths

```